

PetAdopt+ — Full System Documentation

> Written in plain language for first-time readers.

> **Project:** Final Year Project (FYP)

> **Stack:** React (frontend) + Node.js/Express (backend) + MongoDB Atlas (database)

Table of Contents

1. [System Overview](#1-system-overview)
2. [System Architecture](#2-system-architecture)
3. [Complete Workflow](#3-complete-workflow)
4. [Feature-by-Feature Breakdown](#4-feature-by-feature-breakdown)
5. [API Endpoints List](#5-api-endpoints-list)
6. [Database Models](#6-database-models)
7. [Authentication Flow](#7-authentication-flow)
8. [Error Handling](#8-error-handling)
9. [User Roles](#9-user-roles)
10. [How to Run (Viva Demo)](#10-how-to-run-viva-demo)

1. System Overview

What is PetAdopt+?

PetAdopt+ is a web application for pet care services. It brings three main services into one place:

Service	What users can do
Pet Adoption	Browse pets in the shelter, apply to adopt, track application status
Pet Shop	Buy food, toys, and accessories online (cart, checkout, Khalti or cash on delivery)
Pet Hostel	Book a room for their pet when they travel (check-in / check-out dates, online payment)

Who uses it?

Role	Who they are	Main tasks
User (customer)	Pet owners / adopters	Sign up, shop, adopt, book hostel, view orders & adoptions
Admin	System manager	Dashboard, manage pets, products, rooms, staff, orders, adoptions
Staff	Hostel workers	View rooms and bookings, update booking status

Purpose of the system

- Help a pet shelter **manage adoptions** digitally
- Run a small **online pet store** with inventory and payments
- Run a **pet boarding (hostel)** with room availability and bookings
- Give **admins** reports (revenue charts, sales, stats) for business decisions

2. System Architecture

Think of the system as **three layers** that talk to each other:

``

|

```
| BROWSER (User sees the website) |  
| React app – runs at http://localhost:5173 |  
| Pages: Shop, Login, Admin Dashboard, Hostel, etc. |
```

|

```
| HTTP requests (fetch)  
| JSON data + Bearer token for login
```

▼

```
| BACKEND SERVER (Brain of the system) |  
| Node.js + Express – runs at http://localhost:4000 |  
| Routes → Controllers → Database models |
```

|

```
| Mongoose
```

▼

```
|
```

| DATABASE (MongoDB Atlas – cloud) |

| Collections: users, pets, products, orders, bookings, ... |

External services (used when needed):

- Cloudinary – stores pet/product/room images
- Gmail (Nodemailer) – verification & notification emails
- Khalti – online payments for orders and hostel bookings

Frontend structure (/frontend)

Folder / file	Purpose
src/pages/	Full screens (Shop, Login, AdminDashboard, etc.)
src/components/	Reusable pieces (Navbar, ProtectedRoute, SessionChecker)
src/config/api.js	Base URL for all API calls (http://localhost:4000)
src/utils/auth.js	Login token stored in browser, session checks
src/App.jsx	All routes and who can access each page

Backend structure (/backend)

Folder / file	Purpose
index.js	Starts server, connects routes and MongoDB
Routes/	URL paths (e.g. /api/auth/login)
Controller/	Business logic (what happens when API is called)
model/	Database schemas (shape of stored data)
Middleware/	Auth checks, file upload, form validation
config/cloudinary.js	Cloudinary setup (loads only when uploading)
utils/emailService.js	Sends emails

How frontend and backend connect

1. User clicks something in the browser (e.g. "Login").
2. React calls the backend, for example:

```
POST http://localhost:4000/api/auth/login with email and password.
```

1. Backend checks the database and sends back JSON, for example:

```
{ success: true, data: { user, token } } .
```

1. Frontend saves the **token** in `localStorage` and uses it on later requests:

```
Authorization: Bearer <token> .
```

1. Protected pages send this token; backend verifies it before allowing access.

Example: Loading the shop products

- Frontend: GET `http://localhost:4000/api/products`
- Backend: reads products from MongoDB, returns list as JSON
- Frontend: draws product cards on screen

3. Complete Workflow

Example A – User logs in and buys a product

Step	Where	What happens
1	Browser	User opens <code>/login</code> , enters email and password
2	Frontend	Sends <code>POST /api/auth/login</code>
3	Backend	Finds user, checks password, checks email verified
4	Backend	Returns JWT token + user info (name, role)
5	Frontend	Saves token and user in <code>localStorage</code>
6	Browser	User goes to <code>/shop</code> , adds items to cart (cart saved in browser)
7	Frontend	Goes to <code>/checkout</code> , sends <code>POST /api/orders</code> with items and address
8	Backend	Checks stock, creates order, may reduce stock (cash on delivery)
9	Backend	Sends confirmation email (if email is configured)
10	Frontend	If Khalti: redirects to payment; if cash: shows success page

Example B – User applies to adopt a pet

Step	What happens
1	User browses <code>/browse-pets</code> (public)
2	Opens pet details (login required)
3	Fills adoption form at <code>/adopt/:id</code>
4	<code>POST /api/adoptions</code> saves application as pending
5	Admin sees it in Admin Pets Management
6	Admin approves or rejects → email sent to applicant

Example C – Admin views dashboard revenue bar chart

Step	What happens
1	Admin opens /admin-dashboard (admin login required)
2	Frontend calls GET /api/dashboard/stats and GET /api/dashboard/revenue
3	Backend counts users, pets, orders, bookings; groups revenue by month
4	Frontend draws bar chart with Chart.js
5	Admin clicks a bar → GET /api/orders/revenue/:month/:year for that month's details

Example D – Admin uploads a pet image

Step	What happens
1	Admin selects image in “Add Pet” form
2	Frontend sends file to POST /api/upload (multipart form)
3	Backend saves file briefly in backend/uploads/ folder
4	Backend uploads file to Cloudinary cloud storage
5	Backend returns image URL (e.g. https://res.cloudinary.com/...)
6	Admin saves pet with that URL in the image field
7	Shop/pet pages show image using that URL

4. Feature-by-Feature Breakdown

4.1 Cloudinary (Image storage)

What it is: Cloudinary is an online service that stores and serves images. Your app does not store large image files in MongoDB – only the **image link (URL)** is stored in the database.

Upload flow (step by step)

1. **Admin** picks an image when adding/editing a pet, product, or hostel room.
2. **Frontend** sends the image file to:

|

POST /api/upload

(field name must be image)

1. **Multer** (middleware) saves the file temporarily in backend/uploads/ with a unique filename.

2. **UploadController** calls Cloudinary and uploads the file to folder petadopt on Cloudinary.

3. Cloudinary returns:

- secure_url - link to view the image

- public_id - ID used to delete the image later

1. **Frontend** puts secure_url into the form and saves the pet/product/room.

2. **Database** stores only the URL string in the image field.

Where images are stored

Location	What is stored
Cloudinary cloud	Actual image files (in folder petadopt)
MongoDB	Text URL only, e.g. https://res.cloudinary.com/your-cloud/image/upload/...
backend/uploads/	Temporary files during upload (not the permanent store)

How images are shown to users

- Pet list, shop, product details, hostel rooms read image from the database.

- The browser loads the image directly from Cloudinary's URL (like any normal website image).

Delete flow

- `DELETE /api/upload/:publicId` removes the image from Cloudinary.
- Used when admin replaces or removes images.

Configuration

Credentials live in `backend/.env` :

- `CLOUDINARY_CLOUD_NAME`
- `CLOUDINARY_API_KEY`
- `CLOUDINARY_API_SECRET`

If these are missing, upload will fail but the rest of the app still runs.

4.2 Product discounts

How discount is stored

On each **product** in the database:

Field	Meaning	Example
<code>price</code>	Normal price in Rs	<code>1000</code>
<code>discount</code>	Percent off (0–100)	<code>20</code> means 20% off

How the offer price is calculated

Formula:

$$\text{Offer price} = \text{round}(\text{price} \times (1 - \text{discount} / 100))$$

Example: Price = Rs 1000, Discount = 20%

→ Offer price = $1000 \times (1 - 20/100) = 1000 \times 0.8 = \text{Rs } 800$

The backend model also has a virtual field `offerPrice` that uses the same formula.

Where discount is set

- **Admin** → Inventory Management → Add/Edit Product → “Discount %” dropdown.

How the user sees it

Page	What user sees
Shop	Old price crossed out, new price in orange, “20% OFF” badge
Product details	Same styling
Cart	Uses discounted price per item (saved when adding to cart)
Checkout	Subtotal and total use cart prices

How it works in the cart

When user clicks “Add to cart” on the shop:

1. Frontend calculates effective price with the same formula.

2. Cart stores `price` = discounted price and keeps `originalPrice` for display.

Note for viva: Checkout sends `subtotal` and `totalAmount` from the cart (with discounts).

The backend also rebuilds line items from the database using each product’s **catalog**

price field when creating the order. Totals on the order often follow what the frontend sent; line-item prices in the order record reflect the database price at order time.

4.3 Admin dashboard – monthly revenue bar graph

What the chart shows

A **bar chart** with 12 bars (Jan–Dec) for the **current year**.

Each bar height = **total revenue** for that month from:

- **Shop orders** (delivered + paid)
- **Hostel bookings** (not cancelled)

What data is fetched

On page load, the admin dashboard calls three APIs:

API	Purpose
GET /api/dashboard/stats	Summary cards (users, pets, revenue total, etc.)
GET /api/dashboard/revenue	Monthly bars data
GET /api/orders/product-sales	Pie chart – top products sold

How monthly revenue is processed (backend)

GET /api/dashboard/revenue (in DashboardController.js):

1. Takes year (default: current year).
2. Finds **orders** where:

– status = Delivered

– paymentStatus = Paid

– Created in that year

1. Only counts order line items whose **product still exists** in inventory (deleted products are excluded).
2. Finds **hostel bookings** where status is not Cancelled.
3. Groups money by month (1–12).
4. Returns array like:

```
`json`  
  
{  
  "success": true,  
  "data": [  
    { "month": "Jan", "orders": 5000, "bookings": 2000, "total": 7000, "orderCount": 3,  
      "bookingCount": 1 },  
    { "month": "Feb", "orders": 0, "bookings": 1500, "total": 1500, ... }  
  ]  
}
```

How it is displayed (frontend)

AdminDashboard.jsx :

1. Saves `revenueData` from the API.
2. Uses **Chart.js** to draw a bar chart.
3. Each bar uses `m.total (orders + bookings)`.
4. Clicking a bar calls `GET /api/orders/revenue/:month/:year` and shows tables of successful orders and paid bookings for that month.

Pie chart (product sales)

- Data from `GET /api/orders/product-sales` .
- Counts quantity sold per product from paid/delivered orders.

- Uses **current product name** from inventory (not old names on orders).
- Skips deleted products.
- Shows top products in a pie chart.

4.4 Other major features

Pet adoption

- Public browse; apply when logged in.
- Application statuses: pending , approved , rejected , withdrawn .
- Admin can approve with pickup details or reject with notes.
- Emails sent on submit, approve, reject.

Pet hostel

- Admin creates rooms (type, price per day, facilities, image).
- User picks dates; system checks availability.
- Booking statuses: Pending → Confirmed → Checked-In → Checked-Out (or Cancelled).
- Payment: Cash or Khalti.
- Staff/admin update status; emails on status change.

Online payments (Khalti)

- User chooses Khalti at checkout or hostel booking.
- Backend creates payment session via Khalti API.
- User pays on Khalti website, returns to your app.
- Backend **verifies** payment, then marks order/booking as paid and updates stock.

Product reviews

- Logged-in users can rate and comment on products (1 review per user per product).
- Shown on product details page.

Hostel room reviews

- Logged-in users can review hostel rooms after staying.
- Shown on hostel room details page.

Staff management

- Admin creates staff accounts (role = staff).
- Staff log in to staff dashboard, view rooms and bookings, update booking status.

Email notifications

Used for: signup verification, password reset, order confirmation, order status updates, adoption updates, hostel booking confirmation and status updates.

Session checking

SessionChecker on the frontend watches if the login token is still valid and can log the user out if expired.

5. API Endpoints List

Base URL: **http://localhost:4000**

All protected routes need header:

Authorization: Bearer <your_token>

Responses usually look like:

`json`

```
{ "success": true, "data": { ... } }
```

`

```
or on error:
```

```
`json`  
  
{ "success": false, "message": "Error description" }
```

```
`
```

Auth – /api/auth

Method	Path	Who	What it does
POST	/signup	Public	Register new user; sends 6-digit email code
POST	/login	Public	Login; returns token + user
GET	/me	Logged in	Get current user profile
POST	/verify-email	Public	Verify email with code
POST	/resend-verification	Public	Resend verification code
POST	/forgot-password	Public	Send password reset code
POST	/verify-reset-code	Public	Check reset code
POST	/reset-password	Public	Set new password
PUT	/update-profile	Logged in	Update name, phone, address, etc.
PUT	/change-password	Logged in	Change password

Example – Login

Request:

```
`json`
```

POST /api/auth/login

```
{
  "email": "user@example.com",
  "password": "MyPass123!"
}
```

Response (success):

```
`json
{
  "success": true,
  "message": "Login successful",
  "data": {
    "user": { "id": "...", "fullName": "John", "email": "user@example.com", "role": "user" },
    "token": "eyJhbGciOiJIUzI1NiIs..."
  }
}
```

Pets – /api/pets

Method	Path	Who	What it does
GET	/	Public	List all pets
GET	/:id	Public	One pet's details
POST	/	Admin*	Add pet
PUT	/:id	Admin*	Update pet
DELETE	/:id	Admin*	Delete pet

*Pet write routes exist; protect with admin in production usage via admin UI only.



Products – /api/products



Method	Path	Who	What it does
GET	/	Public	List products (supports page , limit , filters)
GET	/alerts/low-stock	Admin	Low stock alert list
GET	/:id	Public	One product
POST	/	Admin	Create product
PUT	/:id	Admin	Update product (including discount)
DELETE	/:id	Admin	Delete product
PATCH	/:id/stock	Admin	Update stock quantity
GET	/:productId/reviews	Public	Get reviews
POST	/:productId/reviews	User	Add review
PUT	/:productId/reviews/:reviewId	User	Edit own review
DELETE	/:productId/reviews/:reviewId	User	Delete own review



Orders – /api/orders

Method	Path	Who	What it does
GET	/	Admin	All orders
GET	/stats/summary	Admin	Order statistics
GET	/my-orders	User	Current user's orders
GET	/product-sales	Admin	Data for pie chart
GET	/revenue/:month/:year	Admin	Month drill-down for bar chart
GET	/:id	Admin	Single order
POST	/	User / guest	Create order
PATCH	/:id/status	Admin	Update order status
PATCH	/:id/cancel	User/Admin	Cancel order
DELETE	/:id	Admin	Delete order

Example – Create order

Request:

```
`json`
```

```
POST /api/orders
```

```
Authorization: Bearer <token>
```

```
{
  "customer": {
    "name": "John",
    "email": "john@example.com",
    "phone": "9800000000",
    "address": "Kathmandu"
  },
  "items": [
    { "product": "product_id_here", "quantity": 2 }
  ],
  "subtotal": 1600,
  "shippingFee": 100,
```

```
"totalAmount": 1700,
"paymentMethod": "Cash on Delivery",
"notes": ""
}
```

Hostel rooms – /api/hostel-rooms

Method	Path	Who	What it does
GET	/	Public	List rooms
GET	/:id	Public	Room details
POST	/:id/check-availability	Public	Check dates available
POST	/	Admin	Create room
PUT	/:id	Admin	Update room
DELETE	/:id	Admin	Delete room
GET/POST/PUT/DELETE	/:roomId/reviews/...	User	Room reviews

Hostel bookings – /api/hostel-bookings

Method	Path	Who	What it does
GET	/my-bookings	User	User's bookings
POST	/	User	Create booking
PATCH	/:id/cancel	User	Cancel booking
GET	/	Staff/Admin	All bookings
GET	/:id	Staff/Admin	One booking
PATCH	/:id/status	Staff/Admin	Update status
POST	/admin	Admin	Admin-created booking
DELETE	/:id	Admin	Delete booking

Adoptions – /api/adoptions

Method	Path	Who	What it does
GET	/my-adoptions	User	User's applications
POST	/	Public/User	Submit application
GET	/user/:email	Public	Applications by email
GET	/	Admin	All applications
GET	/:id	Admin	One application
PUT	/:id/status	Admin	Approve/reject
DELETE	/:id	Admin	Delete application

Dashboard – /api/dashboard (Admin only)

Method	Path	What it does
GET	/stats	Dashboard summary numbers
GET	/activities	Recent bookings, orders, adoptions
GET	/revenue	Monthly revenue for bar chart
GET	/users	Users list with pagination
GET	/rooms-overview	Rooms + active bookings
GET	/bookings-overview	Bookings list

Staff – /api/staff (Admin only)

Method	Path	What it does
GET	/	List staff
GET	/:id	One staff member
POST	/	Create staff account
PUT	/:id	Update staff
DELETE	/:id	Delete staff
PATCH	/:id/status	Enable/disable staff
PATCH	/:id/reset-password	Reset staff password

Upload – /api/upload

Method	Path	What it does
POST	/	Upload image file → returns Cloudinary URL
DELETE	/:publicId	Delete image from Cloudinary

Upload response example:

```
`json
{
  "success": true,
  "message": "Image uploaded successfully",
  "url": "https://res.cloudinary.com/.../image.jpg",
  "publicId": "petadopt/abc123"
}
```

Payments – /api/payments (Khalti)

Method	Path	Who	What it does
POST	/initiate-order	User	Start Khalti payment for order
POST	/initiate-booking	User	Start Khalti payment for booking
POST	/verify-order	User	Confirm order payment after Khalti
POST	/verify-booking	User	Confirm booking payment
POST	/refund-order/:orderId	Admin	Refund order
GET	/order/:orderId	User	Payment details for order

6. Database Models

Database: **MongoDB** (cloud via MongoDB Atlas).

|

Connection string: MONGODB_URI in backend/.env .

|

User

Stores: Account login and profile.

Field	Meaning
fullName, email, password	Login identity (password is encrypted)
role	user , admin , or staff
isVerified	Email verified or not
verificationCode	6-digit signup code (temporary)
phone, address, city, country	Profile
resetPasswordCode	For forgot password flow

Pet

Stores: Animals available for adoption.

Field	Meaning
name, type, breed, age, gender	Pet info
status	Available, Booked, Unavailable
image	Cloudinary URL
vaccinations	List of vaccines and dates
description, size, vaccinated	Extra details

Product

Stores: Shop items.

Field	Meaning
name, category, petType, description	Product info
price	Normal price
discount	Percent off (0–100)
stock, lowStockThreshold	Inventory
status	Available, Unavailable, Out of Stock
image	Cloudinary URL
brand, weight	Optional

Order

Stores: Shop purchases.

Field	Meaning
orderNumber	Auto-generated e.g. ORD-12345678
user	Link to user (if logged in)
customer	Name, email, phone, address
items[]	product, productName, quantity, price, subtotal
subtotal, shippingFee, totalAmount	Money
status	Pending → Processing → Shipped → Delivered (or Cancelled/Returned)
paymentStatus	Pending, Paid, Failed, Refunded
paymentMethod	Cash on Delivery, Khalti, etc.
khaltiPayment	Khalti transaction details

Shipping rule: Free shipping if subtotal $\geq$ Rs 10,000; otherwise Rs 100.

AdoptionApplication

Stores: Adoption form submissions.

Field	Meaning
petId, petName	Which pet
fullName, email, phone, address, age, occupation	Applicant
ownsPets, reason, experience, environment	Screening questions
status	pending, approved, rejected, withdrawn
reviewNotes, reviewedBy, reviewedAt	Admin review

HostelRoom

Stores: Boarding rooms.

Field	Meaning
roomNumber, roomName, roomType	Identity
petType, capacity, pricePerDay	Rules and pricing
facilities[], description, image	Details
status	Available, Occupied, Under Maintenance
currentOccupant	Who is in the room now (if any)

HostelBooking

Stores: Room reservations.

Field	Meaning
bookingNumber	Auto e.g. HB-12345678
user, room	Who booked which room
petDetails	Pet name, type, needs
checkInDate, checkOutDate, numberOfDays	Stay period
totalAmount	Price for whole stay
status	Pending, Confirmed, Checked-In, Checked-Out, Cancelled
paymentStatus, paymentMethod	Payment info
contactInfo	Email, phone, emergency contact

ProductReview

Stores: Star rating + comment for a product (one per user per product).

HostelRoomReview

Stores: Star rating + comment for a hostel room (one per user per room).

7. Authentication Flow

Signup

1.

2.

1. User fills name, email, password on /signup .

2.

2. Backend checks password strength (8+ chars, upper, lower, number, special).

3.

3. Backend creates user with isVerified: false .

4.

4. 6-digit code emailed (valid 10 minutes).

5. User enters code on `/verify-email` .

6. Backend sets `isVerified: true` .

7. User can now log in.

Login

1. User submits email + password.

2. Backend compares password (encrypted).

3. If email not verified → error with `needsVerification: true` .

4. If OK → JWT token created (default expiry 7 days).

5. Frontend stores `token` and `user` in `localStorage`.

Protected routes (frontend)

`ProtectedRoute` wrapper checks:

1. Is there a valid token in `localStorage`?

2. If route needs a role (admin/staff), does user's role match?

3. If not → redirect to login or correct dashboard.

Protected routes (backend)

`protect` middleware:

1. Reads `Authorization: Bearer <token>` .
2. Verifies token with `JWT_SECRET` .
3. Loads user from database.
4. Attaches user to `req.user` for the controller.

Extra middleware:

- `adminOnly` – only `role === 'admin'`
- `staffOrAdmin` – staff or admin (hostel operations)

Example: Calling a protected API

```
`javascript

fetch('http://localhost:4000/api/orders/my-orders', {
  headers: {
    'Authorization': 'Bearer ' + localStorage.getItem('token')
  }
})
```

```
`
```

```
---
```

8. Error Handling

How the backend handles errors

Most controllers use **try / catch**:

1. **Try** to do the work (database, email, payment).
2. If something fails → **catch** block runs.
3. Server sends JSON with `success:false` and a `message` humans can read.
4. HTTP status code tells the type of error:

Status	Meaning	Example
400	Bad request / validation	Missing email, weak password
401	Not logged in / bad token	No Bearer token
403	Not allowed	User trying to open admin page
404	Not found	Product or order does not exist
500	Server error	Database down, unexpected crash

Form validation errors

Routes like signup use **express-validator** + `validate` middleware.

Response example:

```
`json
{
  "success": false,
  "message": "Validation error",
  "errors": [
```

```
{ "field": "email", "message": "Please provide a valid email" }  
]  
}
```

How the frontend handles errors

- Checks `response.success` from API.
- Shows `alert()` or on-screen message with `data.message`.
- Login page handles `needsVerification` to send user to verify email.
- `ProtectedRoute` redirects if session expired.

MongoDB connection errors

If database cannot connect on startup:

- Console shows: MongoDB connection error
- Process exits (server stops)
- Fix: check `MONGODB_URI` in `.env` and internet connection.

Port already in use

- Message: Port 4000 already in use
- Fix: run `npm run kill-port` in backend folder, then `npm run dev` again.

9. User Roles

Role	Frontend entry after login	Main pages
user	Browse pets / shop	Shop, cart, hostel, my orders, my adoptions, profile
admin	/admin-dashboard	All admin management + dashboard charts
staff	/staff-dashboard	Staff rooms, staff bookings

10. How to Run (Viva Demo)

Terminal 1 – Backend

```
` bash
```

```
cd backend  
npm run dev
```

```
Wait for:
```

```
Server running on port 4000
```

```
Connected to MongoDB
```

Terminal 2 – Frontend

```
` bash
```

```
cd frontend
```

```
npm run dev
```

```
Open: http://localhost:5173
```

Demo checklist

- [] Login as **user** – shop, cart, checkout
- [] Login as **admin** – dashboard bar chart + pie chart
- [] Show **pet adoption** flow
- [] Show **hostel booking**
- [] Mention **Cloudinary** for images, **Khalti** for payments, **MongoDB Atlas** for data

If something fails

```
` bash
```

Free port 4000

```
cd backend && npm run kill-port && npm run dev
```

```
` ``
```

Quick Reference — Main URLs

What	URL
Website	http://localhost:5173
Backend API	http://localhost:4000
Login	http://localhost:5173/login
Shop	http://localhost:5173/shop
Admin dashboard	http://localhost:5173/admin-dashboard

Document generated from the PetAdopt+ codebase for FYP documentation and viva preparation.